**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА**

**Факультет електроніки та комп'ютерних технологій**

**Звіт**

про виконання лабораторної роботи №5

"Складова квадратурна формула для чисельного інтегрування. Методи підвищення точності. Адаптивний алгоритм"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

ас. Каськун О. Д.

**Львів-2026**

**Мета роботи:** Реалізувати складову формулу Сімпсона, методи Рунге-Ромберга й Ейткена та адаптивне інтегрування; порівняти їхню точність.

**Теоретичні відомості:**

Складова формула Сімпсона використовує парне число відрізків N і N+1 вузлів, h = (b-a)/N. Її загальний порядок точності - 4. Уточнення Рунге-Ромберга: I_R = S(N0) + [S(N0)-S(N0/2)]/15. Метод Ейткена оцінює порядок за трьома послідовними сітками. Обидва підходи спираються на переважання одного степеневого члена похибки. В адаптивному алгоритмі порівнюю одну панель Сімпсона з двома половинними, оцінюю похибку як |S2-S1|/15 і ділю локальний допуск навпіл під час подальшого поділу.

## Виконання роботи

Репозиторій на GitHub:
https://github.com/RakhimAbdulkhanov/numerical_methods_2026

1. Задав $f(x) = 50 + 20\sin(\pi x/12) + 5\exp[-0.2(x-12)^2]$ на [0; 24]. Її первісна: $F(x) = 50x - (240/\pi)\cos(\pi x/12) + (5\sqrt{\pi}/(2\sqrt{0.2}))\mathrm{erf}(\sqrt{0.2}(x-12))$. За добу синусоїдальна складова інтегрується в нуль. Тому $I0 = 1200 + 5\sqrt{(\pi/0.2)}\mathrm{erf}(12\sqrt{0.2})$.

```python
import math
from dataclasses import dataclass

import matplotlib.pyplot as plt
import mpmath as mp
import numpy as np


A, B = 0.0, 24.0
TARGET = 1e-12
mp.mp.dps = 60
```

```python
def load(x):
    return (50 + 20 * math.sin(math.pi * x / 12)
            + 5 * math.exp(-0.2 * (x - 12)**2))

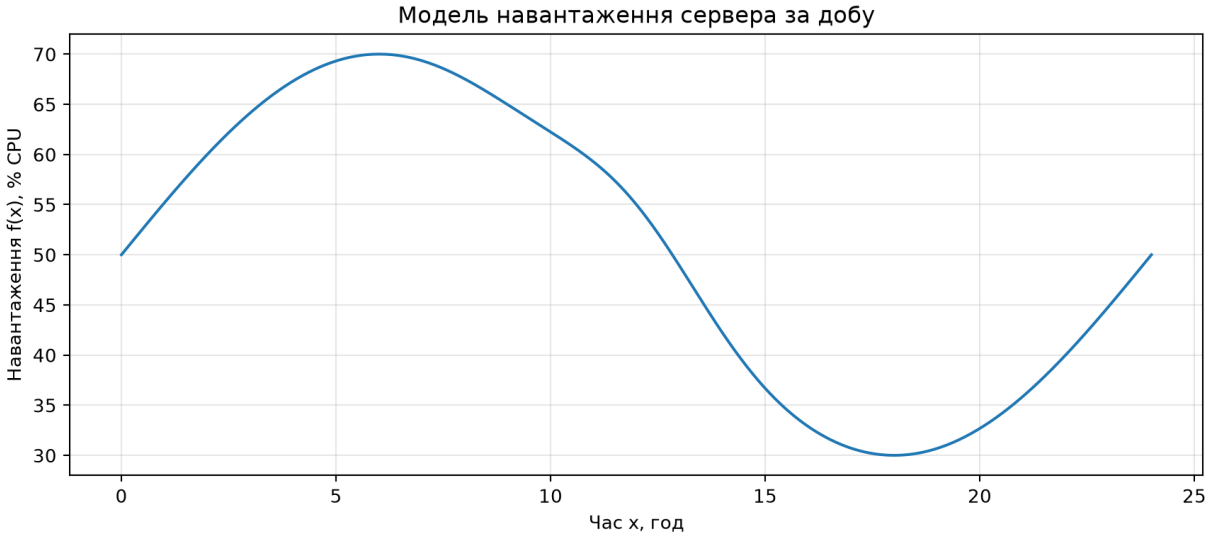
def exact_integral():
    # синус за повну добу дає нуль, гаусіан інтегруєм через erf
    c = mp.mpf("0.2")
    return 1200 + 5 * mp.sqrt(mp.pi / c) * mp.erf(12 * mp.sqrt(c))

def absolute_error(value, exact):
    return float(abs(mp.mpf(value) - exact))
```

| Величина | Значення |
|---|---|
| I0, % CPU · год | 1219.816636488029418505644031754 2 |

Аналітичний вираз обчислив з 60 десятковими знаками через mpmath. Похибки двійкових float порівнював із цим еталоном, а не з округленим float-значенням I0. Це не дає отримати штучний нуль похибки через однакове округлення двох чисел.

```
def make_plots(result):
    fig, ax = plt.subplots(figsize=(9, 4), layout="constrained")
    x = np.linspace(A, B, 500)
    ax.plot(x, [load(t) for t in x])
    ax.set(xlabel="Час x, год", ylabel="Навантаження f(x), % CPU",
           title="Модель навантаження сервера за добу")
    ax.grid(alpha=0.3)
```




2. Реалізував формулу Сімпсона та уточнення. Перевірив усі парні N від 10 до 1000. У формулах N означає число відрізків, хоча в одному пункті завдання його названо числом вузлів.

```
def simpson(function, a, b, n):
    if not isinstance(n, int) or isinstance(n, bool) or n < 2 or n % 2:
        raise ValueError("n має бути додатним парним числом відрізків")
    h = (b - a) / n
    terms = [function(a), function(b)]
    terms.extend((4 if i % 2 else 2) * function(a + i * h)
                 for i in range(1, n))
    return h * math.fsum(terms) / 3

def runge_romberg(fine, coarse):
    return fine + (fine - coarse) / 15

def aitken(fine, medium, coarse):
    first, second = medium - fine, coarse - medium
    denominator = second - first
    if denominator == 0 or first == 0 or second == 0:
        raise ValueError("недостатньо різниці між наближеннями")
    return fine - first * first / denominator, math.log2(abs(second / first))
```

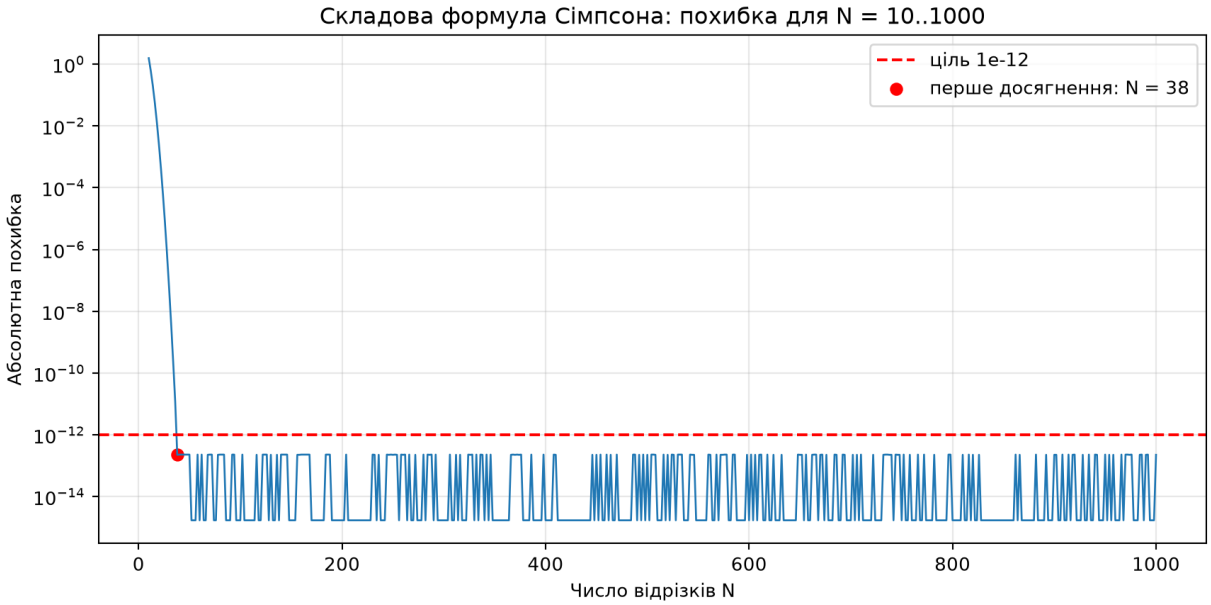
3. Організував пошук першого N з похибкою не більшою за 1e-12 та порівняння сіток. У коді окремо перевірив допустимість N0, щоб не приховувати суперечність умови.

```
def experiment():
    exact = exact_integral()
    counts = list(range(10, 1001, 2))
    values = [simpson(load, A, B, n) for n in counts]
    errors = [absolute_error(value, exact) for value in values]
    n_opt = next((n for n, error in zip(counts, errors)
                  if error <= TARGET), None)
    if n_opt is None:
        raise ArithmeticError("цільову точність не досягнуто до n=1000")
    allowed = [n for n in range(16, n_opt // 8 + 1, 8)]
    # за порожнього діапазону явно використовуєм демонстраційне n0
    n0 = allowed[-1] if allowed else 16
    fine, medium, coarse = [simpson(load, A, B, n0 // k)
                            for k in (1, 2, 4)]
    improved, order = aitken(fine, medium, coarse)
    comparisons = [(f"S({n0})", fine), (f"S({n0 // 2})", medium),
                   (f"S({n0 // 4})", coarse),
                   ("Runge-Romberg", runge_romberg(fine, medium)),
                   ("Aitken", improved)]
    adaptive = [(delta, adaptive_simpson(load, A, B, delta))
                for delta in (1e-2, 1e-4, 1e-6, 1e-8, 1e-10, 1e-12)]
    return dict(exact=exact, counts=counts, errors=errors,
                n_opt=n_opt, opt_error=errors[counts.index(n_opt)],
                n0=n0, n0_compliant=bool(allowed), comparisons=comparisons,
                order=order, adaptive=adaptive)
```

| Параметр | Значення |
|---|---|
| Nopt | 38 |
| Похибка при Nopt | 2.256641e-13 |
| Nopt / 8 | 4.75 |
| Допустимі N0 | немає |
| Демонстраційне N0 | 16 |

У завданні одночасно вимагається 10 <= N0 <= Nopt/8 та кратність N0 числу 8. Для отриманого Nopt = 38 верхня межа дорівнює 4.75, тому виконати ці вимоги неможливо. Далі взяв N0 = 16 лише для демонстрації заданих формул на сітках 16, 8, 4. Це не є допустимим N0 за початковою умовою. Не змінював Nopt і не підмінював функцію, щоб штучно отримати потрібний діапазон.

```
    fig2, ax2 = plt.subplots(figsize=(9, 4.5), layout="constrained")
    ax2.semilogy(result['counts'], result['errors'], linewidth=1)
    ax2.axhline(TARGET, color="red", linestyle="--", label="ціль 1e-12")
    ax2.scatter(result['n_opt'], result['opt_error'], color="red",
                label=f"перше досягнення: N = {result['n_opt']}")
    ax2.set(xlabel="Число відрізків N", ylabel="Абсолютна похибка",
            title="Складова формула Сімпсона: похибка для N = 10..1000")
    ax2.grid(alpha=0.3)
    ax2.legend()
```




Похибка дуже швидко зменшується, а далі коливається біля межі точності float. Синус на рівномірній сітці за повний період взаємно компенсується; у гауссіана крайові похідні дуже малі. Тому на цій конкретній функції не слід очікувати простого закону C $h^4$ на всьому діапазоні N.

4. Реалізував адаптивний алгоритм з повторним використанням уже обчислених значень. Умова зупинки стосується оцінки похибки; фактичну похибку незалежно перевірив за аналітичним еталоном. Лічильник враховує кожне звернення до f(x).

```python
class AdaptiveResult:
    value: float
    estimated_error: float
    evaluations: int
    intervals: int

def adaptive_simpson(function, a, b, delta, max_depth=40):
    if not math.isfinite(delta) or delta <= 0:
        raise ValueError("delta має бути додатним і скінченним")
    if b <= a:
        raise ValueError("потрібно a < b")
    evaluations = 0

    def evaluate(x):
        nonlocal evaluations
        evaluations += 1
        return function(x)

    def panel(left, right, fl, fm, fr):
        return (right - left) * (fl + 4 * fm + fr) / 6

    def refine(left, right, fl, fm, fr, whole, tolerance, depth):
        middle = (left + right) / 2
        q1, q3 = (left + middle) / 2, (middle + right) / 2
        if q1 == left or q3 == right:
            raise ArithmeticError("крок досяг межі машинної точності")
        fq1, fq3 = evaluate(q1), evaluate(q3)
        lower = panel(left, middle, fl, fq1, fm)
        upper = panel(middle, right, fm, fq3, fr)
        correction = (lower + upper - whole) / 15
        if abs(correction) <= tolerance:
            return lower + upper + correction, abs(correction), 1
```

```
        if depth == 0:
            raise ArithmeticError("вичерпано глибину адаптивного поділу")
        lval, lerr, lc = refine(left, middle, fl, fq1, fm, lower,
                                tolerance / 2, depth - 1)
        rval, rerr, rc = refine(middle, right, fm, fq3, fr, upper,
                                tolerance / 2, depth - 1)
        return math.fsum((lval, rval)), lerr + rerr, lc + rc

    fl, fm, fr = evaluate(a), evaluate((a + b) / 2), evaluate(b)
    value, error, intervals = refine(a, b, fl, fm, fr,
                                     panel(a, b, fl, fm, fr), delta, max_depth)
    return AdaptiveResult(value, error, evaluations, intervals)
```

| delta | Фактична похибка | Оцінка похибки | Виклики f |
|---|---|---|---|
| 1e-02 | 5.82650e-04 | 2.9302e-03 | 45 |
| 1e-04 | 1.07899e-07 | 2.4134e-05 | 145 |
| 1e-06 | 7.28052e-10 | 4.5185e-07 | 405 |
| 1e-08 | 2.25664e-13 | 3.1620e-09 | 1337 |
| 1e-10 | 1.70959e-15 | 2.6710e-11 | 4617 |
| 1e-12 | 1.70959e-15 | 4.4743e-13 | 12817 |

5. Вивів реальні результати обчислень. Таблиця містить також демонстраційне порівняння Рунге-Ромберга й Ейткена.

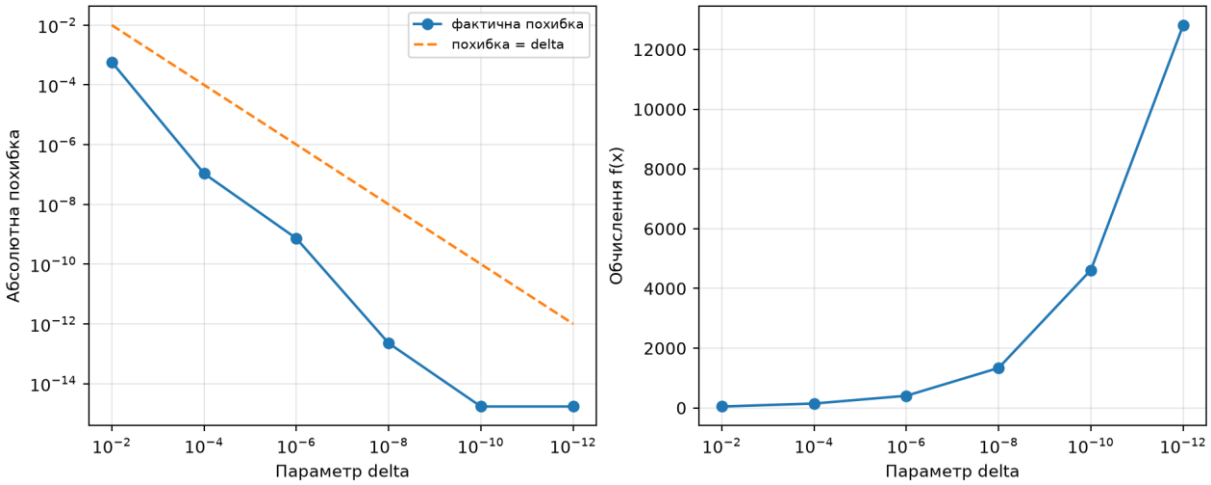
```
def print_results(result):
    print(f"[a, b] = [{A:g}, {B:g}]; target = {TARGET:.0e}")
    print(f"I0 = {mp.nstr(result['exact'], 35)}")
    print(f"Nopt = {result['n_opt']}; error = {result['opt_error']:.6e}")
    if not result['n0_compliant']:
        print(f"No N0: 10 <= N0 <= {result['n_opt'] / 8:g}, N0 % 8 = 0")
        print(f"Demonstration N0 = {result['n0']} (outside prescribed range)")
    print(f"{'method':<15} {'integral':>21} {'absolute error':>15}")
    for name, value in result['comparisons']:
        print(f"{name:<15} {value:21.13f} "
              f"{absolute_error(value, result['exact']):15.6e}")
    print(f"p = {result['order']:.9f}")
    print(f"{'delta':>9} {'integral':>20} {'actual error':>13} "
          f"{'estimate':>12} {'calls':>6}")
    for delta, row in result['adaptive']:
        print(f"{delta:9.0e} {row.value:20.12f} "
              f"{absolute_error(row.value, result['exact']):13.5e} "
              f"{row.estimated_error:12.4e} {row.evaluations:6d}")
```

```
PS C:\Users\ponti> python numerical_methods_2026/lab5/main.py
[a, b] = [0, 24]; target = 1e-12
I0 = 1219.816636488029418505644031754204 2
Nopt = 38; error = 2.256641e-13
No N0: 10 <= N0 <= 4.75, N0 % 8 = 0
Demonstration N0 = 16 (outside prescribed range)
method                      integral  absolute error
S(16)             1219.7617263381467    5.491015e-02
S(8)              1216.6268909304745    3.189746e+00
S(4)              1220.0597268646763    2.430904e-01
Runge-Romberg     1219.9707153653248    1.540789e-01
Aitken            1218.2654284082337    1.551208e+00
p = 0.131011211
    delta            integral  actual error     estimate  calls
    1e-02    1219.816053838302   5.82650e-04   2.9302e-03     45
    1e-04    1219.816636380130   1.07899e-07   2.4134e-05    145
    1e-06    1219.816636487301   7.28052e-10   4.5185e-07    405
    1e-08    1219.816636488030   2.25664e-13   3.1620e-09   1337
    1e-10    1219.816636488029   1.70959e-15   2.6710e-11   4617
    1e-12    1219.816636488029   1.70959e-15   4.4743e-13  12817
PS C:\Users\ponti> _
```

На грубих сітках N = 4, 8, 16 уточнення не покращили результат. Оцінка p = 0.131011 не наближається до 4: ці сітки не перебувають у режимі переважання одного члена C $h^4$. Тому тут некоректно робити висновок, що сам метод Сімпсона має порядок 0.131. Екстраполяція не гарантує покращення поза її асимптотичними припущеннями.

6. Побудував залежності фактичної похибки та кількості викликів f(x) від delta. Зі зменшенням допуску потрібно більше обчислень; фактична похибка зрештою впирається в округлення. delta - параметр контролю оцінки, а не точна наперед відома фактична похибка.

```python
    fig3, axes = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
    tolerances = [delta for delta, _ in result['adaptive']]
    actual = [absolute_error(row.value, result['exact'])
              for _, row in result['adaptive']]
    calls = [row.evaluations for _, row in result['adaptive']]
    axes[0].loglog(tolerances, actual, "o-", label="фактична похибка")
    axes[0].loglog(tolerances, tolerances, "--", label="похибка = delta")
    axes[0].set(xlabel="Параметр delta", ylabel="Абсолютна похибка")
    axes[0].legend(fontsize=8)
    axes[1].semilogx(tolerances, calls, "o-")
    axes[1].set(xlabel="Параметр delta", ylabel="Обчислення f(x)")
    for axis in axes:
        axis.invert_xaxis()
        axis.grid(alpha=0.3)
    return [fig, fig2, fig3]
```




Перевірка реалізації. Запустив автоматичні тести: правильність базових формул, перевірку некоректних параметрів та узгодженість результатів експерименту.

```python
def main():
    result = experiment()
    print_results(result)
    make_plots(result)
    plt.show()
```

```python
import math
import unittest

import mpmath as mp

import main


class IntegrationTests(unittest.TestCase):
    def test_simpson_polynomials(self):
        for degree in range(4):
            expected = 2**(degree + 1) / (degree + 1)
            self.assertAlmostEqual(main.simpson(lambda x: x**degree,
                                                0, 2, 10), expected)

    def test_invalid_counts(self):
        for n in (0, 1, 3, 2.0, True):
            with self.assertRaises(ValueError):
                main.simpson(main.load, 0, 24, n)

    def test_reference_against_independent_quadrature(self):
        f = lambda x: (50 + 20 * mp.sin(mp.pi*x/12)
                       + 5 * mp.exp(-mp.mpf('0.2')*(x-12)**2))
        independent = mp.quad(f, [0, 6, 12, 18, 24])
        self.assertLess(abs(independent - main.exact_integral()), mp.mpf('1e-50'))

    def test_adaptive_function_count_and_accuracy(self):
        calls = []
        def function(x):
            calls.append(x)
            return math.exp(x)
        result = main.adaptive_simpson(function, 0, 1, 1e-9)
```

```
        self.assertEqual(result.evaluations, len(calls))
        self.assertEqual(len(calls), len(set(calls)))
        self.assertLess(abs(result.value - math.expm1(1)), 1e-9)
        self.assertLessEqual(result.estimated_error, 1e-9)
        self.assertEqual(result.evaluations, 4 * result.intervals + 1)

    def test_adaptive_limits(self):
        with self.assertRaises(ValueError):
            main.adaptive_simpson(main.load, 0, 24, 0)
        with self.assertRaises(ArithmeticError):
            main.adaptive_simpson(math.exp, 0, 1, 1e-20, max_depth=0)

    def test_extrapolation(self):
        self.assertAlmostEqual(main.runge_romberg(1.01, 1.16), 1)
        value, order = main.aitken(1.01, 1.16, 3.56)
        self.assertAlmostEqual(value, 1)
        self.assertAlmostEqual(order, 4)

    def test_experiment_constraints_and_actual_error(self):
        result = main.experiment()
        self.assertLessEqual(result['opt_error'], main.TARGET)
        for n, error in zip(result['counts'], result['errors']):
            if n < result['n_opt']:
                self.assertGreater(error, main.TARGET)
        self.assertFalse(result['n0_compliant'])
        self.assertLess(result['n_opt'] / 8, 10)
        for delta, row in result['adaptive']:
            self.assertLess(main.absolute_error(row.value, result['exact']), delta)
            self.assertLessEqual(row.estimated_error, delta)


if __name__ == '__main__':
```

```
1  unittest.main()
```

```
PS C:\Users\ponti> python -m unittest discover -s numerical_methods_2026/lab5 -p test_main.py -v
test_adaptive_function_count_and_accuracy
(test_main.IntegrationTests.test_adaptive_function_count_and_accuracy) ... ok
test_adaptive_limits (test_main.IntegrationTests.test_adaptive_limits) ... ok
test_experiment_constraints_and_actual_error
(test_main.IntegrationTests.test_experiment_constraints_and_actual_error) ... ok
test_extrapolation (test_main.IntegrationTests.test_extrapolation) ... ok
test_invalid_counts (test_main.IntegrationTests.test_invalid_counts) ... ok
test_reference_against_independent_quadrature
(test_main.IntegrationTests.test_reference_against_independent_quadrature) ... ok
test_simpson_polynomials (test_main.IntegrationTests.test_simpson_polynomials) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.547s

OK
PS C:\Users\ponti> _
```

## Висновок:

Реалізував складову формулу Сімпсона й отримав I0 = 1219.816636488029 % CPU · год. Ціль 1e-12 вперше досягнута при N = 38 з похибкою 2.257e-13. Виявив несумісність вимог до N0 та окремо показав демонстраційні обчислення. На грубих сітках екстраполяції погіршили результат, що показує важливість їхніх асимптотичних передумов. Адаптивний алгоритм на всіх перевірених допусках дав фактичну похибку нижчу за delta; менший допуск потребував більшої кількості обчислень функції.